LAPORAN TUGAS

STURKTUR DATA DAN ALGORITMA
PENJELASAN TAMBAHAN TUGAS 2 STACK

Dosen Pengampu:

Prof. Dr. Taufik Fuadi Abidin, M.Tech

Disusun oleh:

Muhammad Aqil Mubarak (250810701100003)

JURUSAN INFORMATIKA

FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM

UNIVERSITAS SYIAH KUALA

BANDA ACEH

2026

PROBLEM TUGAS 2

Poin 1

Membuat program untuk menyelesaikan masalah validasi ekspresi tanda kurung yang seimbang (balanced parentheses) seperti "()", "{}", "[]" menggunakan stack yang diimplementasikan menggunakan array.

Poin 1, anda dimintakan untuk membuat program yang menerima sebuah input berupa tanda kurung dan memeriksa apakah ekspresi tanda kurung tersebut seimbang. Contoh ekspresi:

(({{{[[( ) ( )]]}}})) – contoh ekspresi tanda kurung yang seimbang.
([{{{[[( ) ( )]]}}})) – contoh ekspresi tanda kurung yang tidak seimbang.

Masukan : Ekspresi tanda kurung.
Luaran : Kesimpulan apakah ekspresi seimbang atau tidak seimbang.
Implementasi : Struktur data stack menggunakan array.

Poin 2

Mengubah ekspresi aritmatika infix ke postfix menggunakan struktur data stack yang diimplementasikan menggunakan linked list. Dapat menggunakan struct yang dibahas saat praktikum terkait stack.

Poin 2, mengubah ekspresi aritmatika infix yang memiliki tanda kurung seimbang ke dalam bentuk ekspresi aritmatika postfix.

Masukan : Ekspresi aritmatika infix.
Luaran : Ekspresi aritmatika postfix.
Implementasi : Struktur data stack menggunakan linked list.

Poin 3

Menentukan nilai akhir dari ekspresi aritmatika postfix menggunakan stack yang diimplementasikan menggunakan array dan linked list. Dapat menggunakan struct yang dibahas saat praktikum terkait stack.

Poin 3, menentukan nilai akhir dari ekspresi aritmatika postfix yang diperoleh dari Poin 2 dengan menggunakan struktur data stack.

Masukan : Ekspresi aritmatika postfix.
Luaran : Nilai dari hasil perhitungan ekspresi aritmatika postfix tersebut.
Implementasi : Struktur data stack menggunakan array dan linked list.

PENJELASAN TAMBAHAN

Poin 1

Proses validasi kurung pada fungsi “isMatch” ini bertujuan untuk memastikan setiap tanda kurung buka “(, {, [” memiliki pasangan tutup yang cocok dan dengan urutan yang benar. Dalam implementasi menggunakan Stack berbasis Array, dengan jumlah memori yang dibutuhkan untuk pengecekan ini bersifat statis dan terbatas sesuai panjang input. Logika utamanya adalah memasukkan (*push*) setiap tanda kurung buka ke dalam *stack* dan mengeluarkan (*pop*) serta mencocokkannya saat bertemu tanda kurung tutup menggunakan fungsi “isMatch”. Jika pada akhir penelusuran *stack* berada dalam keadaan kosong, maka ekspresi dinyatakan seimbang.

```c
// Fungsi untuk membuat validasi ekspresi tanda kurung
int isMatch(char open, char close) {
    if (open == '(' && close == ')') return 1;
    if (open == '{' && close == '}') return 1;
    if (open == '[' && close == ']') return 1;

    return 0;
}
```

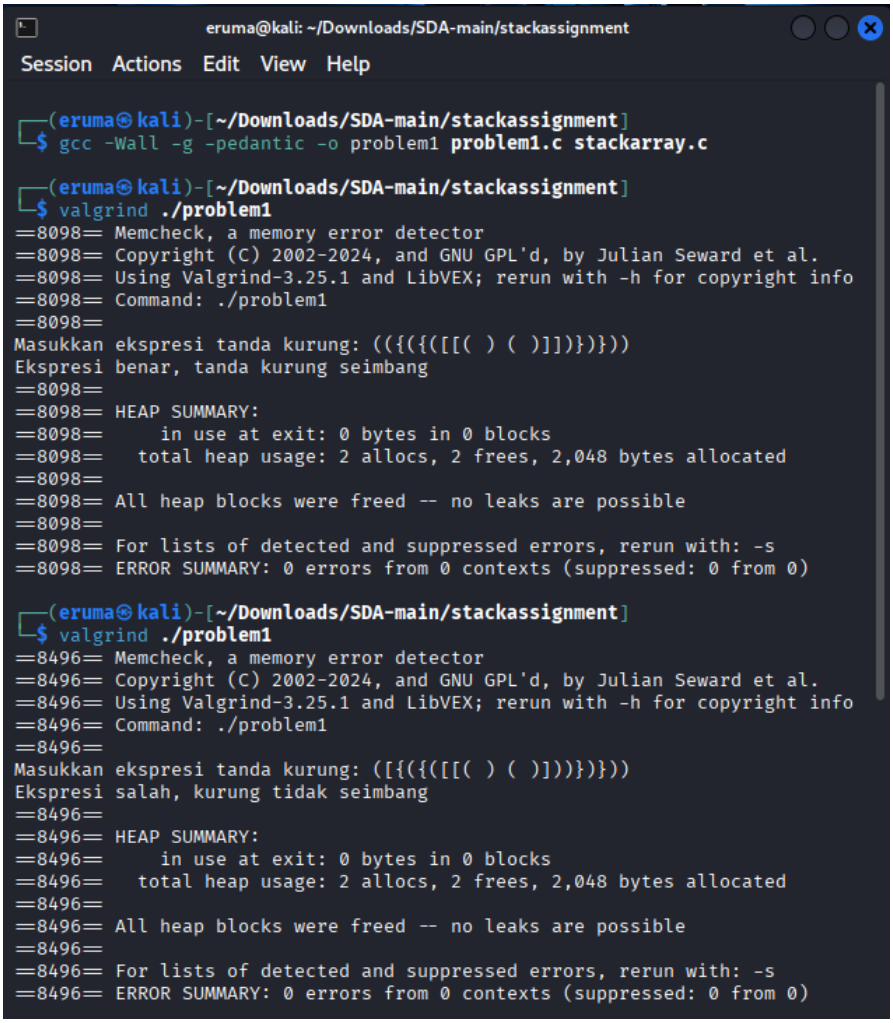
Gambar 1.1 Fungsi “isMatch”

```c
for (int i = 0; i < panjang; i++) {
        if (ekspresi[i] == '(' || ekspresi[i] == '{' || ekspresi[i] == '[' ) {
            pushArr(&st, ekspresi[i]);
        }
        else if (ekspresi[i] == ')' || ekspresi[i] == '}' || ekspresi[i] == ']') {
            // Kondisi jika tanda kurung tidak lengkap pada bagian close
            if (isEmptyArr(st)) {
                printf("Ekspresi salah, tanda kurung tutup tidak ada\n");
                return 0;
            }
            char open = popArr(&st);

            // Kondisi jika ekspresi tanda kurung berlebih
            if (!isMatch(open, ekspresi[i])) {
                printf("Ekspresi salah, kurung tidak seimbang\n");
                return 0;
            }
        }
    }
```

Gambar 1.2 Logika penelusuran karakter

Demikian, program pada Problem 1 ini telah berhasil memberikan output validasi yang akurat dalam menentukan keseimbangan berbagai jenis tanda kurung melalui proses penelusuran karakter yang konsisten, dari inputan yang akan membandingkan kurung-kurung

tersebut apakah seimbang atau tidak seimbang, yang menggunakan struktur data *stack*. Implementasi Stack berbasis Array ini dengan mengalokasi memori statis, di mana data disimpan dalam array lokal dengan ukuran tetap MAX yang didefinisikan secara permanen pada file header. Karena seluruh data disimpan dalam memori *stack* sistem dan tidak menggunakan alokasi memori dinamis seperti "malloc", maka memori tersebut akan dibebaskan secara otomatis oleh sistem segera setelah ruang lingkup fungsi berakhir. Hal ini memastikan bahwa program beroperasi dengan sangat efisien tanpa adanya risiko kebocoran memori (*memory leak*) karena tidak ada sumber daya pada *heap* yang tertinggal tanpa dibebaskan secara manual, yang sudah di validasi melewati pengecekan memori menggunakan valgrind.



Gambar 1.3 *Output* problem 1

Poin 2

Implementasi *stack linked list* pada problem ke 2 ini ada fungsi bernama "priority" untuk menentukan urutan operator, di mana operator perkalian serta pembagian diberikan nilai 2, sedangkan penjumlahan dan pengurangan diberikan nilai 1. Kemudian adapun fungsi "infixTopostfix" yang menjalankan algoritma penelusuran karakter per karakter, di mana setiap operan yang terdeteksi melalui fungsi "isalnum" akan langsung dimasukkan ke dalam array hasil, sementara operator akan dikelola melalui *stack* berbasis *linked list*. Logika pada kode juga menangani tanda kurung secara spesifik; tanda kurung buka akan di-*push* ke dalam *stack*, sementara tanda kurung tutup akan memicu proses *pop* semua operator di dalam *stack* hingga bertemu pasangan kurung bukanya, memastikan urutan operasi matematika tetap terjaga sesuai kaidah standar.

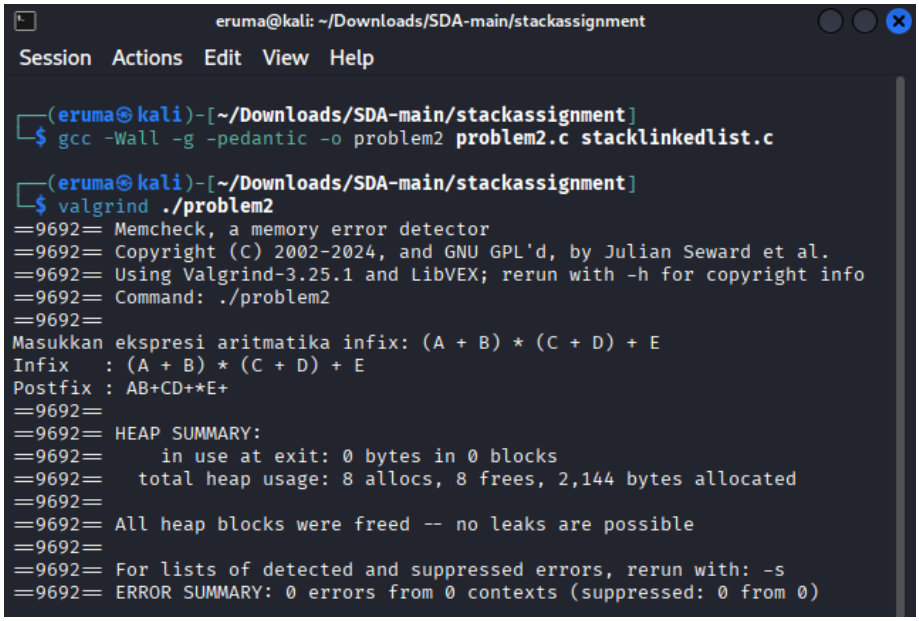
```c
// Fungsi priority untuk prioritas operator aritmatika
int priority(char operator) {
    int pick = 0;
    // Prioritas tinggi
    if (operator == '*' || operator == '/') {
        pick = 2;
    }
    // Prioritas rendah
    else if (operator == '+' || operator == '-') {
        pick = 1;
    }

    return pick;
}
```

Gambar 2.1 Fungsi "priority"

```c
// Kondisi untuk mengecek karakter operan atau angka yang sedang dibaca
        if (isalnum(ekspresi)) {
            postfix[j++] = ekspresi;
        }
        // Kondisi kurung terbuka maka akan di push
        else if (ekspresi == '(') {
            pushLL(&ps, ekspresi);
        }
        // Kondisi kurung tertutup maka akan di pop
        else if (ekspresi == ')') {
            while (!isEmptyLL(&ps) && getLL(&ps) != '(') {
                postfix[j++] = popLL(&ps);
            }
            // Menghapus kurung tutup tanpa di print
            popLL(&ps);
        }
        else {
            // Memindahkan operator dari stack ke postfix jika prioritasnya lebih tinggi atau sama
            while (!isEmptyLL(&ps) && priority(getLL(&ps)) >= priority(ekspresi)) {
                postfix[j++] = popLL(&ps);
            }
            // Mem-push operator baru ke dalam stack
            pushLL(&ps, ekspresi);
        }
```

Gambar 2.2 Logika penelusuran karakter

Demikian, hasil *output* pada program menunjukkan keberhasilan konversi ekspresi aritmatika dari bentuk infix (A+B)*C-D/E menjadi bentuk postfix AB+C*DE/- secara akurat. Dalam proses ini, program secara otomatis mengatur urutan operan dan operator berdasarkan derajat prioritas serta keberadaan tanda kurung, di mana operasi penjumlahan (A+B) diproses terlebih dahulu karena berada di dalam kurung, diikuti oleh perkalian dengan C, pembagian D/E, dan diakhiri dengan operasi pengurangan sebagai operator dengan prioritas terendah dalam ekspresi tersebut. Penggunaan spasi pada input juga berhasil diabaikan oleh logika program sehingga menghasilkan string postfix yang bersih dan siap untuk dievaluasi pada tahap berikutnya. Terakhir, program berhasil berjalan sukses tanpa ada *memory leaked* dibuktikan dengan *output* yang di verifikasi menggunakan valgrind



Gambar 2.3 *Output* program 2

Poin 3

Pada Problem 3 ini dibuat secara modular yang menggabungkan dari stack berbasis array dan stack berbasis linked list, untuk mengevaluasi ekspresi aritmatika “postfix” dengan dukungan penuh terhadap angka multi-digit serta penggunaan spasi sebagai pemisah antar karakter. Adapun fungsi “operation” yang bertugas menangani eksekusi operator dasar seperti penjumlahan, pengurangan, perkalian, dan pembagian, sementara fungsi evaluation menggunakan teknik *string tokenizing* melalui “strtok” untuk memecah string input menjadi potongan token individu. Setiap token angka kemudian dikonversi menjadi nilai integer menggunakan fungsi atoi sebelum dimasukkan ke dalam tumpukan. Kemudian struktur logika di dalam fungsi evaluasi ini memungkinkan pemilihan implementasi *stack* secara fleksibel antara StackArr (berbasis Array) atau StackLL (berbasis Linked List) sesuai instruksi pengguna, sementara bagian main mengelola alur pengambilan input serta pengukuran durasi eksekusi menggunakan fungsi clock() untuk tujuan analisis performa.

```c
// Fungsi operation untuk menentukan tiap eksekusi operator numerik
int operation(int a, int b, char operation) {
    switch (operation) {
        case '+':
        return a + b;
        case '-':
        return a - b;
        case '*':
        return a * b;
        case '/':
        return a / b;
        default:

        return 0;
    }
}
```

Gambar 3.1 Fungsi “Operation”

```c
// Kondisi pertama implementasi stack Array
    if (choice == 1) {
        Stackarr ps;
        initStackArr(&ps);
        // Looping untuk memproses setiap karakter dalam string postfix dengan implementasi stack array
        while (token != NULL) {
            if (isdigit(token[0])) {
                pushArr(&ps, atoi(token));
            }
            else if (strchr("+-*/", token[0])) {
                op2 = popArr(&ps);
                op1 = popArr(&ps);
                pushArr(&ps, operation(op1, op2, token[0]));
            }
            token = strtok(NULL, " ");
        }
        return popArr(&ps);
    }
```

Gambar 3.2 Alur kondisi Implementasi stack array

```c
// Kondisi kedua implementasi stack linkedlist
} else {
    StackLL ps;
    initStackLL(&ps);
    // Looping untuk memproses setiap karakter dalam string postfix dengan implementasi stack linkedlist
    while (token != NULL) {
        if (isdigit(token[0])) {
            pushLL(&ps, atoi(token));
        }
        else if (strchr("+-*/", token[0])) {
            op2 = popLL(&ps);
            op1 = popLL(&ps);
            pushLL(&ps, operation(op1, op2, token[0]));
        }
        token = strtok(NULL, " ");
    }
    return popLL(&ps);
}
```

Gambar 3.3 Alur kondisi Implementasi stack linked list

Berdasarkan hasil *output* terminal, program berhasil mengevaluasi ekspresi postfix seperti 15 3 2 - 4 10 + * 7 / * dengan hasil akhir yang sangat akurat yaitu 30, baik menggunakan implementasi Array maupun Linked List. Walaupun durasi waktu yang tercatat menampilkan nilai 0.000000 karena kecepatan pemrosesan CPU yang jauh melampaui beban kerja ekspresi pendek, hal ini membuktikan bahwa efisiensi algoritma yang diterapkan. Mengenai aspek keamanan memori, program dipastikan bebas dari kebocoran memori (*no memory leak*) karena pada implementasi Array, memori dikelola secara statis pada tumpukan sistem, sedangkan pada implementasi Linked List, setiap node yang dialokasikan secara dinamis selalu dibebaskan kembali melalui pemanggilan fungsi "free(temp)" di dalam operasi "popLL". Karena setiap operan yang masuk akhirnya akan di "*pop*" untuk proses perhitungan atau pengambilan hasil akhir, seluruh sumber daya memori yang pernah dialokasikan dipastikan telah dikembalikan ke sistem sepenuhnya, yang dibuktikan melalui pengecekan menggunakan valgrind.

```
eruma@kali: ~/Downloads/SDA-main/stackassignment
Session  Actions  Edit  View  Help
┌──(eruma㉿kali)-[~/Downloads/SDA-main/stackassignment]
└─$ gcc -Wall -g -pedantic -o problem3 problem3.c stackarray.c stacklinkedlis
t.c

┌──(eruma㉿kali)-[~/Downloads/SDA-main/stackassignment]
└─$ valgrind ./problem3
==11238== Memcheck, a memory error detector
==11238== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==11238== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==11238== Command: ./problem3
==11238==
Masukkan ekspresi postfix: 15 3 2 - 4 10 + * 7 / *
Pilih Implementasi Stack:
1. Array
2. LinkedList
1
=== Hasil Evaluasi ===
Postfix  : 15 3 2 - 4 10 + * 7 / *
Hasil    ; 30
Waktu    : 0.003660
==11238==
==11238== HEAP SUMMARY:
==11238==     in use at exit: 0 bytes in 0 blocks
==11238==   total heap usage: 2 allocs, 2 frees, 2,048 bytes allocated
==11238==
==11238== All heap blocks were freed -- no leaks are possible
==11238==
==11238== For lists of detected and suppressed errors, rerun with: -s
==11238== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Gambar 3.4 *Output* program 3 Array

```
┌──(eruma㉿kali)-[~/Downloads/SDA-main/stackassignment]
└─$ valgrind ./problem3
==11837== Memcheck, a memory error detector
==11837== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==11837== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==11837== Command: ./problem3
==11837==
Masukkan ekspresi postfix: 15 3 2 - 4 10 + * 7 / *
Pilih Implementasi Stack:
1. Array
2. LinkedList
2
=== Hasil Evaluasi ===
Postfix  : 15 3 2 - 4 10 + * 7 / *
Hasil    ; 30
Waktu    : 0.003221
==11837==
==11837== HEAP SUMMARY:
==11837==     in use at exit: 0 bytes in 0 blocks
==11837==   total heap usage: 13 allocs, 13 frees, 2,224 bytes allocated
==11837==
==11837== All heap blocks were freed -- no leaks are possible
==11837==
==11837== For lists of detected and suppressed errors, rerun with: -s
==11837== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Gambar 3.5 *Output* program 3 linkedlist